

INGEGNERIA, GESTIONE ED EVOLUZIONE SOFTWARE

EVOLUZIONE DI SMARTBUGS

Meccanismo Intelligente di Selezione dei Tool

Luciano Bercini

0522501684

Maggio 2026

| Contesto del Progetto

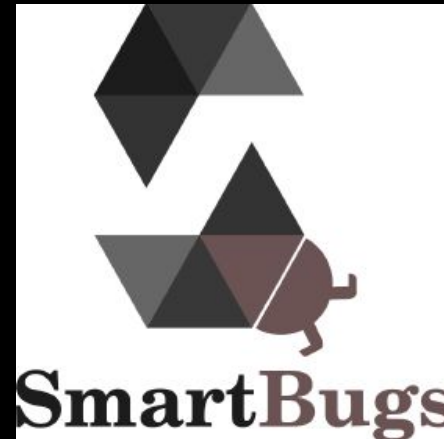
SmartBugs è un framework estensibile per l'analisi automatizzata delle vulnerabilità negli smart contract Ethereum.

Il sistema offre l'esecuzione di tutti i tool disponibili o di un sottoinsieme scelto manualmente dall'utente.

Il Problema

Seppur flessibile, SmartBugs non ha un meccanismo intelligente per selezionare lo strumento più appropriato in base alle caratteristiche dello smart contract in input. Questo comporta:

- Inefficienza computazionale (overhead inutile)
- Analisi ridondanti o non pertinenti
- Mancanza di adattabilità alle caratteristiche del codice
- Necessità di esperienza da parte dell'utente



| Soluzione

Introduzione di una modalità intelligente di selezione dello strumento più pertinente in base alle caratteristiche del contratto in input, andando ad utilizzare dei modelli di machine learning già addestrati.

Più nello specifico, la soluzione propone:

- Integrazione di SCsVulLyzer per l'estrazione delle feature dei contratti.
- La selezione del sottoinsieme rilevante di queste feature per i modelli.
- L'esecuzione dei modelli di machine learning (ognuno specializzato per una determinata categoria di vulnerabilità).
- Aggregazione dei risultati per la costruzione automatica di una lista di tool da eseguire.
- L'invocazione di SmartBugs con i tool identificati.

Vincoli della modifica

- Modularità
- Backward compatibility
- Minimizzazione di modifiche all'attuale pipeline

Fase di Reverse Engineering

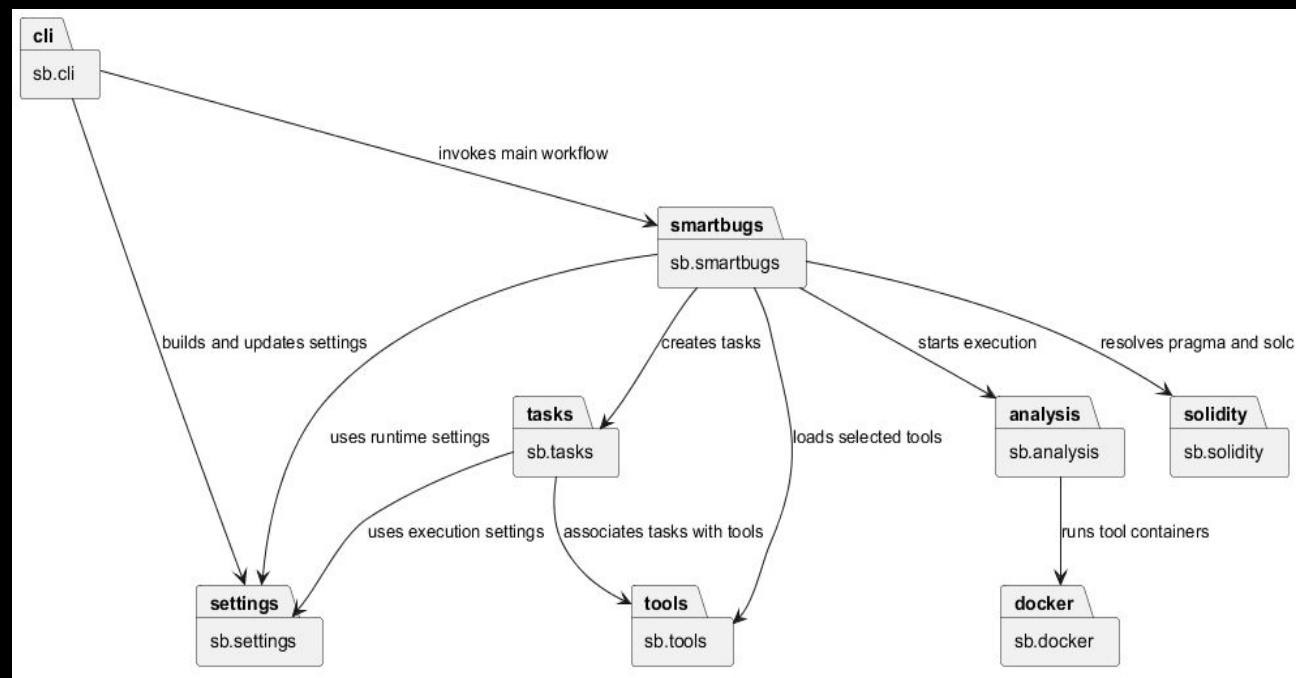
Comprendere l'architettura, identificare i componenti chiave e valutare come la modifica proposta possa essere integrata.

Panoramica Architetture

SmartBugs segue un'architettura modulare in Python, orchestrata dal modulo `sb.smartbugs`.

Moduli Chiave:

- `sb.cli`: Entry point e parsing argomenti.
- `sb.settings`: Gestione configurazioni globali.
- `sb.tools/tasks`: Astrazione di tool e unità di esecuzione.
- `sb.docker`: Interfaccia con Docker.
- `sb.solidity`: Risoluzione compilatore e pragma.
- `sb.analysis`: Esecuzione task.



Package Diagram ottenuto ispezionando i moduli e le loro interazioni (import), enfatizzando il ruolo di orchestrazione di `sb.smartbugs`.

Astrazioni Core del Sistema



Task

Unità fondamentale di esecuzione. Bridge tra file di input, tool selezionato e configurazioni globali.



Tool

Modella un singolo analizzatore. Incapsula metadati (versione, immagine Docker) e logica di invocazione.



Settings

Configurazione globale. Gestisce parametri di runtime, limiti di CPU/Memory e directory dei risultati.

Dependency Matrix

Quantificazione delle interazioni tra i moduli core per identificare più in profondità le relazioni.

Modulo (Da \ A)	cli	settings	smartbugs	tools	tasks	analysis	docker	solidity
cli	0	3	2	0	0	0	0	0
settings	0	0	0	0	0	0	0	0
smartbugs	0	3	0	3	3	2	3	5
tasks	0	2	0	2	0	0	0	0
analysis	0	2	0	0	4	0	2	0

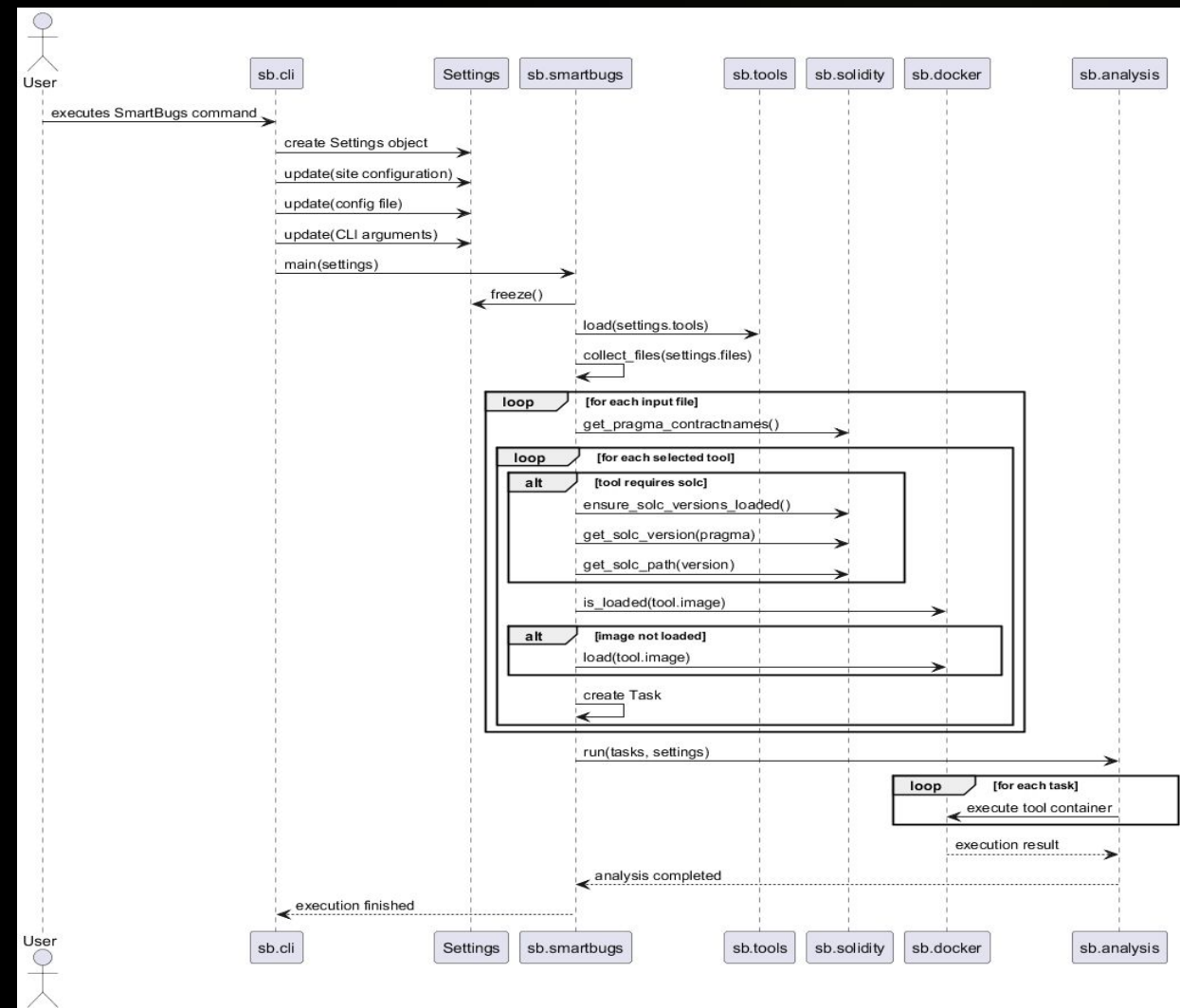
Risultato: sb.smartbugs emerge come l'orchestratore centrale con il più alto grado di dipendenze.

Workflow di Esecuzione Attuale

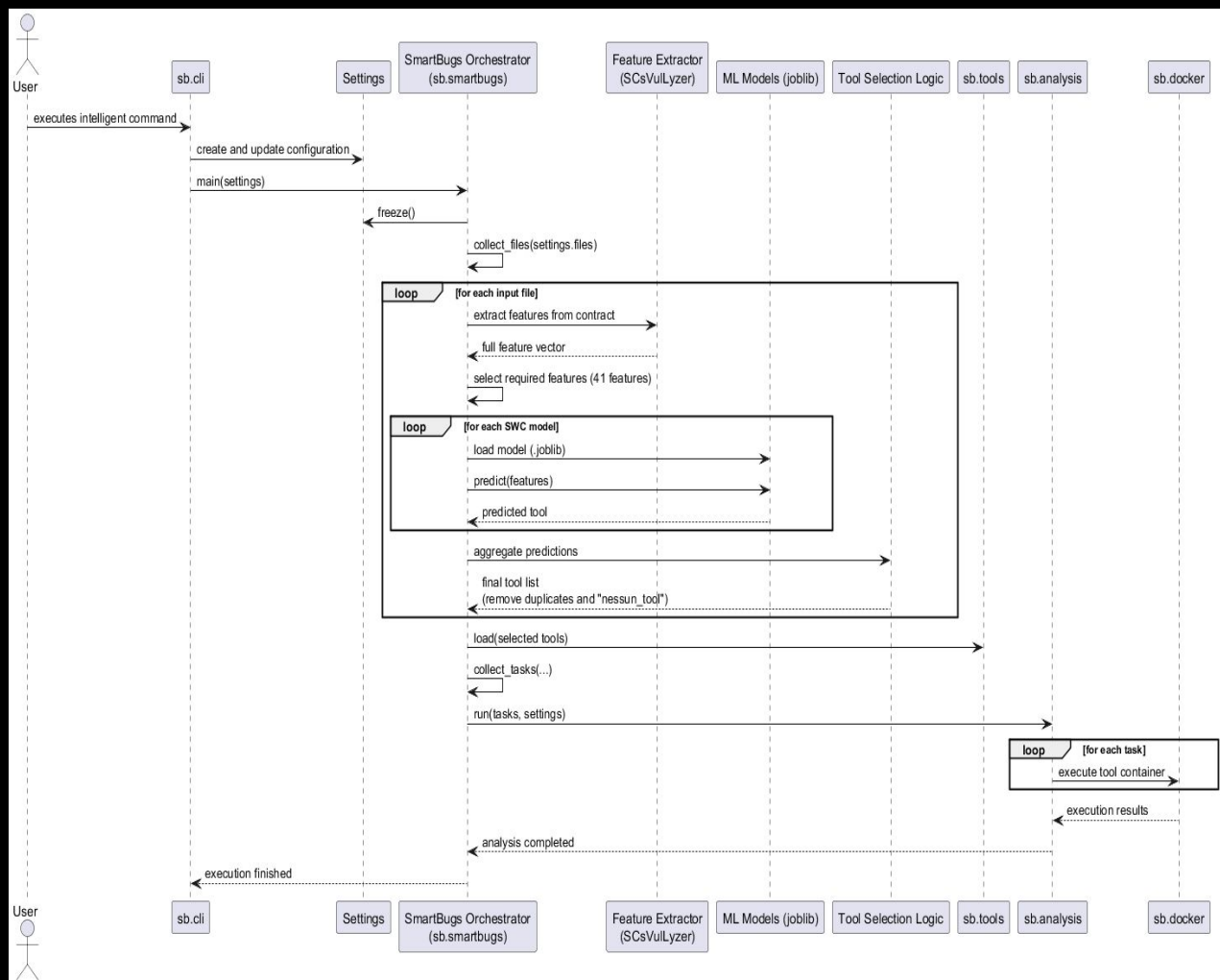
L'analisi del sequence diagram ha rivelato una rigida sequenza lineare:

- Inizializzazione: CLI -> Settings.
- Preparazione: Caricamento tool specificati e scansione file.
- Task Generation: Creazione 1-to-1 tra file e tool scelti.
- Execution: Orchestrazione container Docker tramite sb.analysis.

Punto di estensione identificato: Subito dopo l'inizializzazione, prima di caricare i tool.



Nuovo Workflow Intelligente



Il flusso evoluto automatizza la decisione:

- 1 Estrazione feature (AST, Opcodes, Bytecode)
- 2 Selezione del sottoinsieme richiesto (41 feature)
- 3 Esecuzione dei modelli ML
- 4 Aggregazione e filtraggio dei tool predetti
- 5 Invocazione pipeline originale SmartBugs

Vantaggio: Risparmio risorse analizzando solo ciò che è necessario.

Evoluzione: Selezione Intelligente



CR 1: Features

Integrazione di un modulo di estrazione feature (SCsVulLyzer) per l'estrazione automatica delle feature rilevanti dagli smart contract.



CR 2: ML Models

Integrazione di 8 modelli pre-trained (joblib) per predire il tool ottimale per ogni categoria di vulnerabilità.



CR 3: Workflow

Integrazione dei due precedenti moduli nella pipeline di SmartBugs tramite un nuovo comando intelligente.

Fase di Impact Analysis

Valutazione della propagazione delle modifiche

Risultati dell'Analisi di Impatto su CR3

Riportiamo il caso più complesso (CR3) perché gli altri due sono semplicemente moduli aggiunti senza modifiche alla pipeline.

CR 3: Identificazione Starting Impact Set (SIS)

Il SIS è stato identificato tramite un'analisi della change request che è andata a mappare i concetti introdotti dalla CR sui moduli esistenti. Questa analisi è stata supportata dal package diagram e dal class diagram, insieme al codice sorgente.

- sb.cli: la linea di comando deve essere modificata per accettare i nuovi parametri (e.g. la flag di smart execution).
- sb.settings: il modulo deve essere ampliato per contenere anche i nuovi parametri di runtime.
- sb.smartbugs: la logica di orchestrazione deve essere ampliata per eseguire l'estrazione feature e l'inferenza dei modelli prima di passare alla sua normale esecuzione.

Risultati dell'Analisi di Impatto su CR3

Riportiamo il caso più complesso (CR3) perché gli altri due sono semplicemente moduli aggiunti senza modifiche alla pipeline.

CR 3: Identificazione Candidate Impact Set (CIS)

Il CIS è stato derivato dal SIS attraverso la Matrice delle Dipendenze, seguita da un'ispezione manuale del codice sorgente per evitare la sovrastima dei moduli impattati (over-estimation).

L'analisi delle dipendenze suggeriva un possibile impatto su **sb.tasks**, che dipende da **sb.settings**. L'ispezione manuale ha però rivelato che la classe Task agisce come puro aggregatore di dati (tool, file e settings) e che quindi non era necessario includerlo nel CIS.

Grazie a questo filtro, il CIS identificato ha coinciso esattamente con i moduli del SIS. A valle della fase di sviluppo, l'Actual Impact Set (AIS) ha confermato pienamente la stima iniziale.

Recall = 100%
Precision = 100%

| Strategia di Testing

La strategia di Testing combina test di unità, di componente, di integrazione e di regressione per garantire sia la correttezza delle nuove funzioni sia la stabilità del sistema originale.

Criteri di adeguatezza: I criteri adottati sono di tipo funzionale e orientati al cambiamento (change-oriented), focalizzandosi sulla copertura dei Change Request piuttosto che sulla copertura strutturale esaustiva.

Infrastruttura: È stata riutilizzata e allargata l'infrastruttura di test esistente basata su pytest.

Progettazione dei Test (Category Partition)

Metodologia: È stata utilizzata la tecnica della Category Partition per derivare casi di test rappresentativi per il nuovo workflow intelligente.

Parametrizzazione: Sono stati definiti parametri chiave come l'input del contratto, lo stato dell'estrazione delle feature, le predizioni del modello e la modalità di esecuzione.

Test Post-Modifica: Sono stati derivati 10 nuovi casi di test (da TC-POST-01 a TC-POST-10) per validare le casistiche di successo, gestione errori e filtraggio dei tool.

Test Pre-Modifica: Gli 11 casi di test originali dell'orchestrazione (da TC-PRE-01 a TC-PRE-11) sono stati preservati e classificati come "Unchanged" per la regressione.

Category Name	Category Values
Contract Input	C1: Valid Solidity contract C2: Solidity contract with syntax errors C3: Empty or invalid file
Feature Extraction	F1: Features correctly extracted F2: Partial feature extraction with missing values F3: Feature extraction failure
Feature Selection	S1: All required model features are available S2: Some required model features are missing S3: Feature subset cannot be constructed
Model Prediction	M1: Single valid tool predicted M2: Multiple valid tools predicted M3: Duplicate tools predicted M4: Invalid or unknown tool predicted M5: <code>nessun_tool</code> predicted
Tool Selection	T1: One valid tool selected T2: Multiple valid tools selected T3: Empty tool list after filtering
Execution Mode	E1: Intelligent mode enabled E2: Manual mode enabled
Intelligent Configuration	G1: Valid intelligent workflow configuration G2: Missing intelligent workflow configuration G3: Invalid intelligent workflow configuration

Relation	Constraint
Contract Input → Feature Extraction	C1 → F1 or F2 C2 → F3 C3 → F3
Feature Extraction → Feature Selection	F1 → S1 F2 → S2 or S3 F3 → no feature selection possible
Feature Selection → Model Prediction	S1 → M1, M2, M3, M4, or M5 S2 → prediction may fail or produce partial results S3 → no prediction possible
Model Prediction → Tool Selection	M1 → T1 M2 → T2 M3 → duplicates removed, then T1 or T2 M4 → invalid tools filtered M5 → removed from the final tool list
Tool Selection → Execution	T1, T2 → SmartBugs execution pipeline is invoked T3 → no task is generated or fallback behavior is applied
Configuration → Execution Mode	G1 → E1 can be configured correctly G2 → E1 may use default values G3 → configuration error is reported
Execution Mode	E1 → intelligent workflow is executed E2 → original manual workflow is executed

Test Frame ID	Category Combination	Expected Behavior
TF-POST-01	C1, F1	Features are extracted successfully from a valid Solidity contract.
TF-POST-02	C2, F3	Feature extraction fails and the error is reported.
TF-POST-03	F1, S1	The required subset of model features is correctly selected.
TF-POST-04	S1, M1, T1	A single valid tool is predicted and selected.
TF-POST-05	S1, M2, T2	Multiple valid tools are predicted and selected.
TF-POST-06	S1, M3, T1/T2	Duplicate tool predictions are removed.
TF-POST-07	S1, M5, T3	<code>nessun_tool</code> is removed from the final tool list.
TF-POST-08	T1/T2, E1	SmartBugs executes the selected tools using the existing pipeline.
TF-POST-09	E2	The original manual workflow remains available and unchanged.
TF-POST-10	G1, E1	The intelligent workflow command-line options are parsed correctly and stored in the runtime configuration.

Risultati Testing

Test di Regression: Verificare che l'orchestrazione originale di SmartBugs (collezione file, caricamento tool, generazione task) non subisse alterazioni quando la modalità intelligente è disabilitata.

Esito: Il 100% dei test di regressione eseguiti ha avuto successo.

Validazione Nuove Funzionalità (CR 1, 2 e 3): Sono stati eseguiti 64 test totali (Unitari e di Integrazione) sui nuovi moduli sb.features, sb.models, sb.cli e sulla nuova logica in sb.smartbugs.

Test di Integrazione: Verificata con successo l'intera pipeline logica: dal parsing dei nuovi parametri CLI (--smart-select), all'estrazione, fino all'aggregazione delle predizioni ML e all'inserimento nel workflow.

Esito: Tutti i 64 test sono passati con successo.

Grazie per l'attenzione